GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

SpecArtisans：面向AI编程的可验证Spec-Driven开发范式

SpecArtisans: A Verifiable Spec-Driven Development Paradigm for AI Coding



刘子夕 Zixi Liu

华为2012实验室编程语言实验室高级工程师

Senior Engineer, Programming Language Lab, 2012 Laboratories, Huawei



GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

SpecArtisans
面向 AI 编程的可验证规格驱动开发

刘子夕
华为 2012 实验室 · 编程语言实验室技术专家



CONTENTS

Part 01 背景介绍：AI 时代的软件开发新挑战

Part 02 技术方案：可验证的规格驱动开发

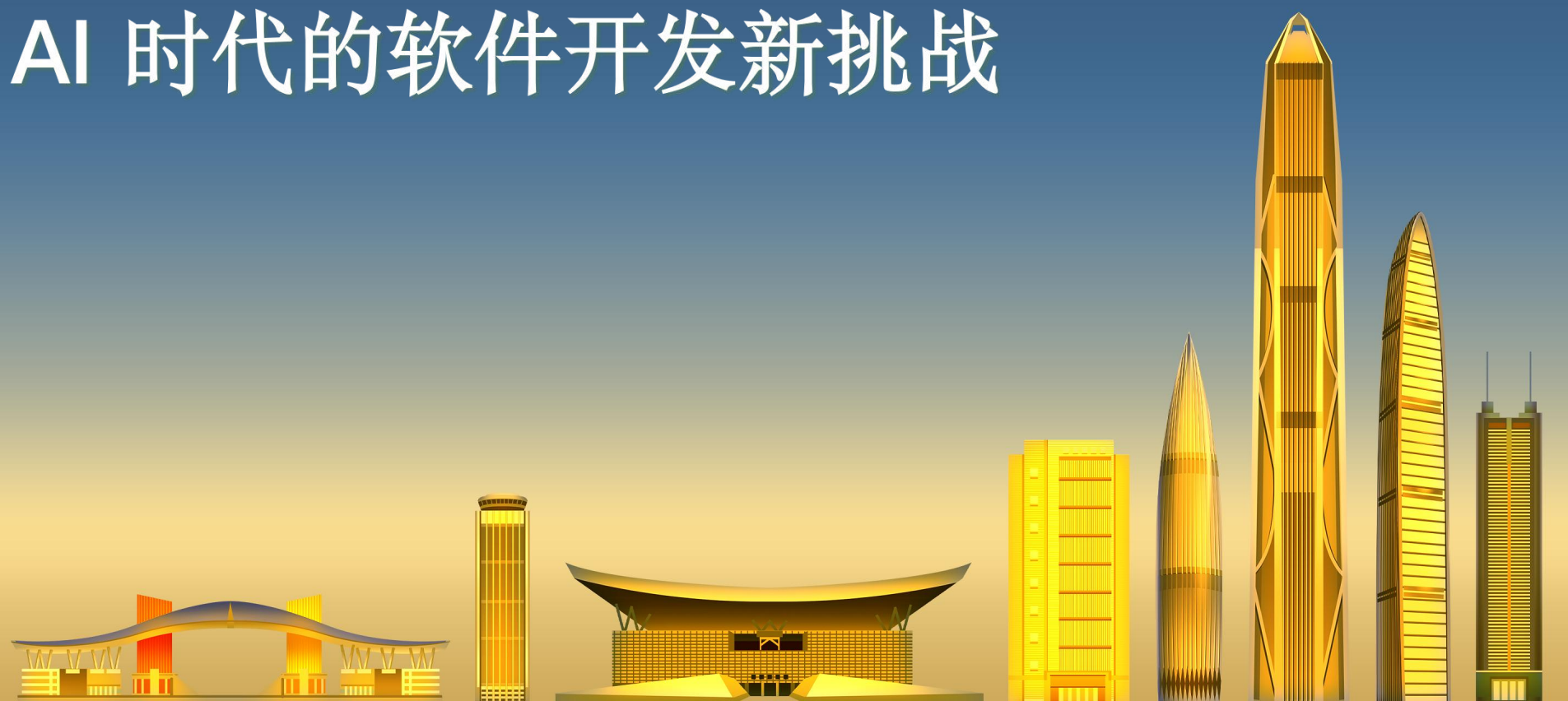
Part 03 实践验证：典型案例与实验结果

Part 04 总结展望：AI 时代的软件工程演进



01

背景介绍：AI 时代的软件开发新挑战

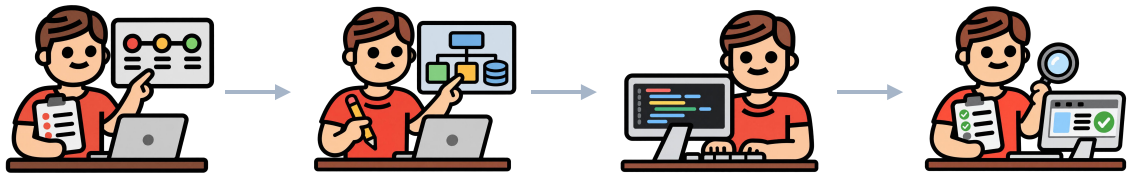


AI 编程改变了软件开发的协作方式

任务执行交给 Agent 后，需求确认与结果判断仍需要共同依据

传统软件开发

不同角色通过需求、设计、代码与测试接力交付



产品经理
提出需求

系统架构师
确定开发方案

开发工程师
实现代码

测试工程师
测试与验收

一群承担不同角色的人

Vibe Coding

人直接向 LLM 表达需求，Agent 完成开发



人

Agent

人和 Agent 协同开发

协作的关键：如何表达意图、检查过程并确认结果？

人和 Agent 需要可检视的协作依据

需求理解：把隐含意图变成明确约定

存量 App 增量开发：同一个收藏需求，贯穿后续设计、实现与验收

01 需求理解

用户的一句话

**“增加收藏功能，
重新打开后仍然保留。”**

看似明确的功能，仍包含未确认的决定

Agent 可能自行补全含义，
把假设带入后续设计和实现。

实现之前，还需要确认

1 适用范围

本地保留，还是跨账号、跨设备同步？

2 操作与异常

取消收藏、重复点击、保存失败如何反馈？

3 验收预期

重启后哪些状态必须保持？怎样判定成功？

未确认的假设，可能会成为设计与实现中的意图偏差

架构设计与产码：变更边界与实现一致性

同一收藏需求：既要融入已有系统，也要完整落实到各层代码

02 架构设计

新增功能，应该改哪里？

复用哪个服务与存储？
状态由谁管理，接口如何衔接？
哪些模块需要修改，哪些应保持不变？

可能偏差

另建一套收藏存储，
绕开原有状态管理，扩大修改范围。

03 代码实现

设计要求，是否完整落地？

UI、状态和持久化是否遵循同一设计？
已确认的行为与异常分支是否遗漏？
不同模块是否共享一致的接口约定？

可能偏差

按钮显示“已收藏”，
持久化却未接通，重启后记录消失。

风险会叠加：架构边界失守，或已确认的意图在产码中丢失

04 测试验收：Agent 说“写完了”，怎样确认功能真正实现?

按已确认的需求，运行完整用户旅程

1 点击收藏 → 显示已收藏

2 退出并重新进入 → 收藏状态仍保留

3 取消收藏 → 恢复未收藏状态

4 再次退出并进入 → 取消结果仍保留

验收依据是否可靠?

从用户需求确定检查项，
避免沿着现有实现编测试，
把同一项遗漏带进验收。

结果是否有运行证据?

界面变化与状态保持都要验证。
编译通过、页面出现，
不足以代表完整旅程通过。

失败能否定向修复?

关联对应需求、设计与代码，
定位原因，限定修改范围，
修复后再验证并检查回归。

缺少需求关联的执行证据，验收与修复就难以形成闭环

可靠交付的三项技术挑战

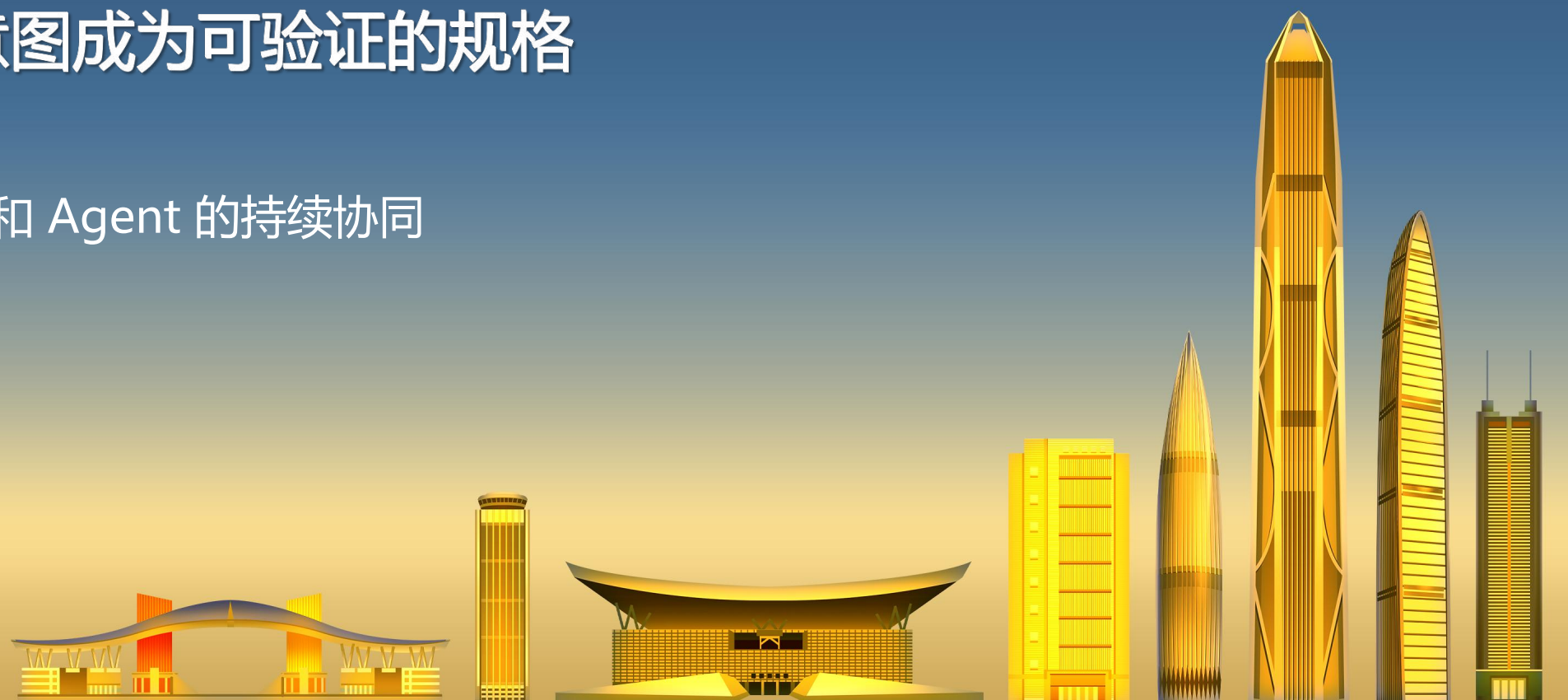
开发过程中的共性挑战	SpecArtisans
意图跨阶段偏离或丢失 需求理解、设计与产码： 如何保留已确认意图，并发现传递遗漏？	**原子需求 ID 贯穿全链路** 关联 PRD、架构变更、代码实现和测试结果， 为人工检视与问题定位保留依据。
变更偏离架构，跨层行为不一致 架构设计与产码： 如何控制变更范围，并对齐各层实现？	**架构感知的增量设计与协同产码** 先理解存量代码，再明确架构变更与代码骨架， 协同约束 UI、接口和业务实现。
缺少用户旅程验证与修复闭环 测试验收与修复： 如何以运行证据定位偏差、修复并复验？	**Spec 双向验证与定向修复** 结合契约检查与 GUI 用户旅程验收， 以验证证据定位偏差，驱动定向修复。

我们的设计 **结构化的规格描述为每一步保留约束与验证依据**

02

技术方案：让意图成为可验证的规格

从意图形式化，到人和 Agent 的持续协同



可靠 AI 编码的瓶颈，不是代码生成能力，而是意图形式化

如何将非正式意图转化为可检查、可演进、可验证的规格

意图形式化光谱



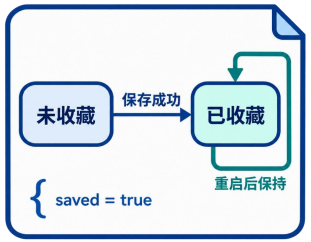
模糊的自然语言

表达目标与愿望



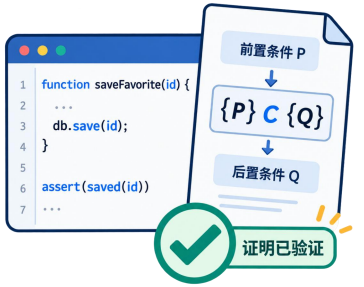
结构化、半形式化规范

明确场景、规则与约束



精确的逻辑图示

显式描述状态与分支



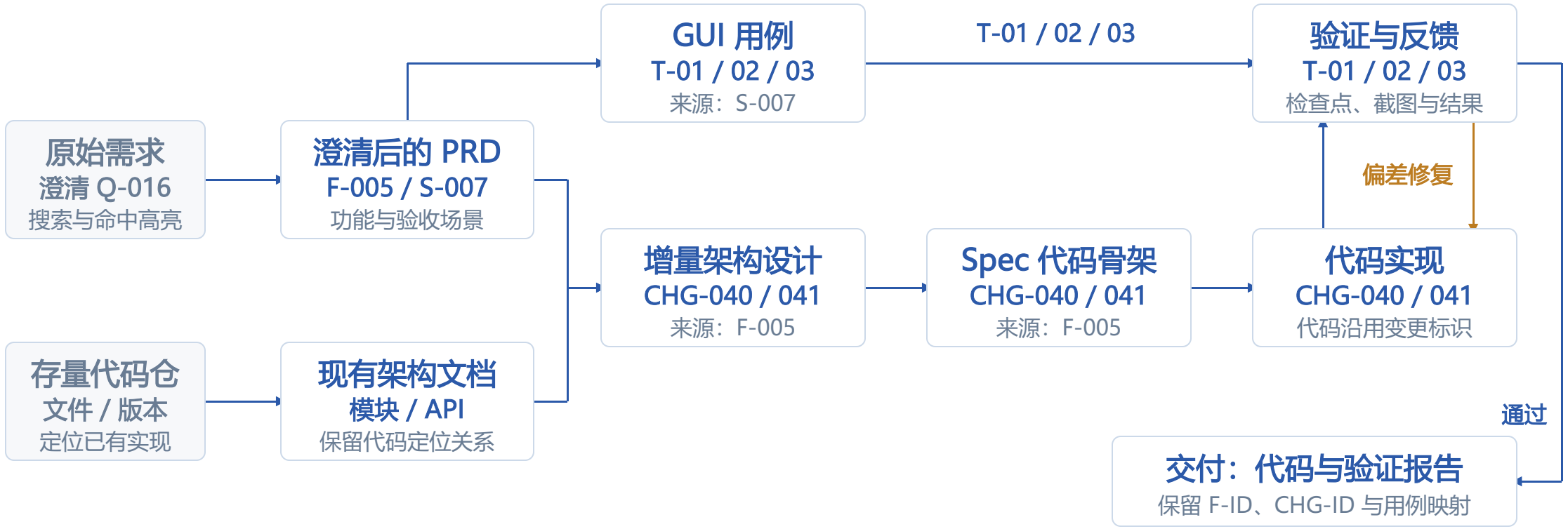
可执行、可验证的代码

实现行为并接受验证

意图严谨度增加

SpecArtisans 关注：让意图在逐步精确的过程中持续保真

以证书搜索为例，功能、变更和用例标识随产物持续传递



来源关系写在各份产物中，可按编号逐级回查

追踪机制：把来源映射写入中间产物

对应上一页的 F-005 证书搜索功能，展开一条追踪链



可检视　　需求是否遗漏？　变更依据是什么？　哪条证据支撑验收？

人或检视 Agent 沿同一映射检查，必要时暂停、回溯与纠偏

真实片段：简历页新增“资格证书”区域

01 功能规则与边界

属性	PRD 内容摘要
功能	展示本地已保存的证书名称 位置在“项目经验”之后、简历页底部
界面规则	标题为“资格证书”，右侧显示添加入口 标签按选择顺序展示
边界	全部展示，随页面滚动 无证书时保留区域，不显示标签
关联项	页面 P-001，UI-S-001 场景 S-001 / S-002

02 页面 P-001 的控件定义

控件 ID	内容与位置	可见性
cert-section-title	“资格证书” 区域顶部左侧	恒显示
cert-tag-list	已保存的证书名称 标题下方流式排列	已有保存记录 时显示

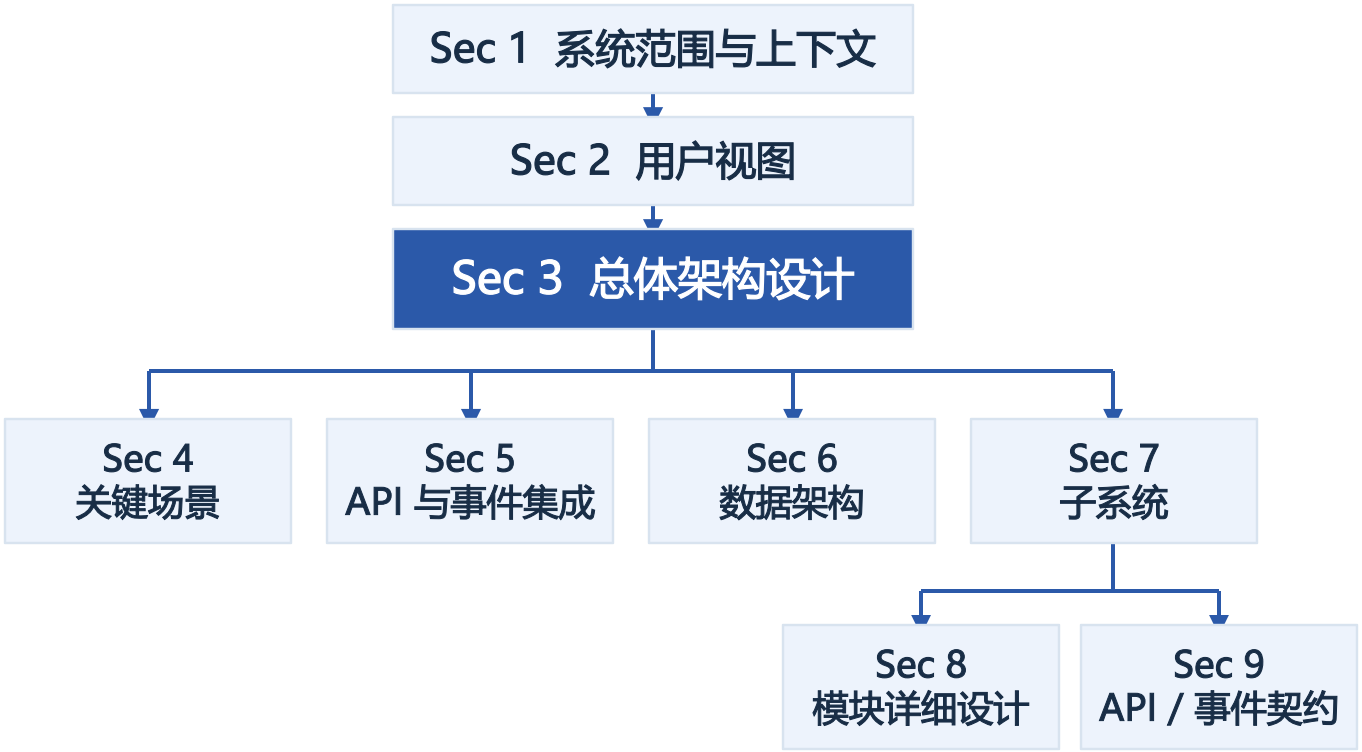
03 场景验收 S-001

GIVEN	已在 P-001，且至少保存一个证书
WHEN	查看资格证书区域 cert-section
THEN	**cert-tag-list 按选择顺序全部展示**
AND	显示标题 cert-section-title 和添加入口

功能关联页面与控件，场景明确操作及可观察的验收条件

架构文档：分层理解系统，约束增量设计

01 从系统上下文逐层定位到模块与契约



02 从文档到设计增量

01 **MAP.md 导览**

按任务加载相关场景、模块和契约

02 **PRD 与存量架构合并**

形成 CHG-ID，明确复用 / 修改 / 新增

03 **评审后约束产码**

更新概要设计，再生成代码骨架

架构分层提供上下文索引，设计增量明确后续实现边界

行为规约让代码骨架可检查

01 生成、规范化与交叉检视

Author

生成代码骨架
写入自然语言行为注释

Formatter

将注释转换为
标准化规约格式

Reviewer

机械校验：语法与规约关键词
LLM检视：与系统设计的一致性

检视发现偏差，回修后再次检查

02 规约直接附着于接口与状态

```
export interface Greeting { message: string; }
export class GreetingService {
  greetedCount: number = 0;
  /* 名字非空时返回包含该名字的问候语，累计问候次数加一。
  @behavior
    @changedBy example-minimal-arkts CHG-01
    @src requirements/greeting.md#greet
    @requires name.size > 0
    @ensures result.message.contains(name)
    @ensures this.greetedCount == OLD(this.greetedCount) + 1
    @effects LOG(INFO, greetingCreated, name.size)
  */
  greet(name: string): Greeting;
}
```

规约字段

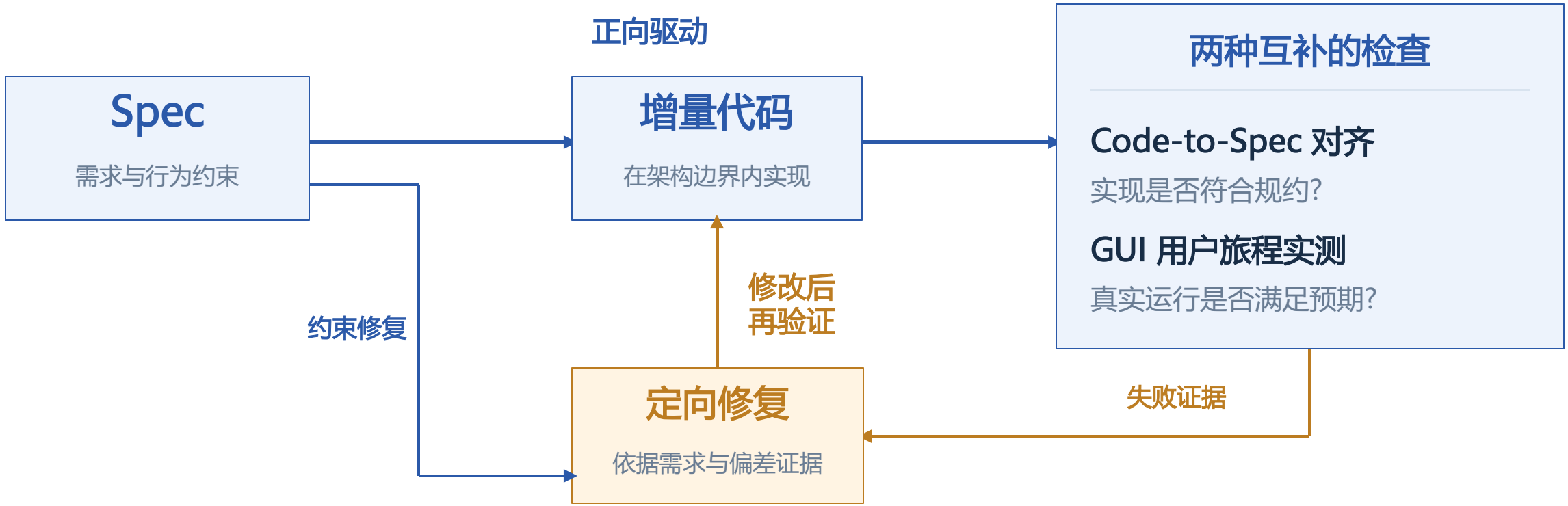
@requires	调用前置条件
@ensures	执行后的结果与状态
@effects	允许的可观察副作用

谓词表示例

谓词	语义	谓词	语义
READ	读取持久化资源	WRITE	向指定位置写入值
UPSERT	存在则更新，否则创建	TRANSACTION	同一事务中执行一组操作
……	……	……	……

双向验证：用偏差证据驱动定向修复

正向规定行为，反向检查实现是否仍忠于意图



失败可定位到原始约束，修复后沿同一标准复验

当前落点：鸿蒙存量 App 的增量迭代

在既有架构内新增功能，检查跨模块协作与原有功能回归

01 可扩展的开发任务

开发模式	有 UI 交互的应用	无 UI 交互的系统
增量开发	**当前重点：鸿蒙存量 App 功能迭代** **复用页面、状态与业务服务**	API 服务、后台任务的功能迭代 验证接口变更与兼容性
全量生成	从 PRD 建立架构与端到端功能 如鸿蒙 App、Web 应用	从业务规格生成服务或工具 如 API、CLI、数据处理任务
代码迁移	如安卓应用迁移至鸿蒙 检验用户旅程的行为一致性	如框架升级、技术栈替换 保持接口语义与业务规则

02 后续扩展方向

多技术栈适配

复用规格与追踪机制
适配产码、构建和测试工具

团队协作与治理

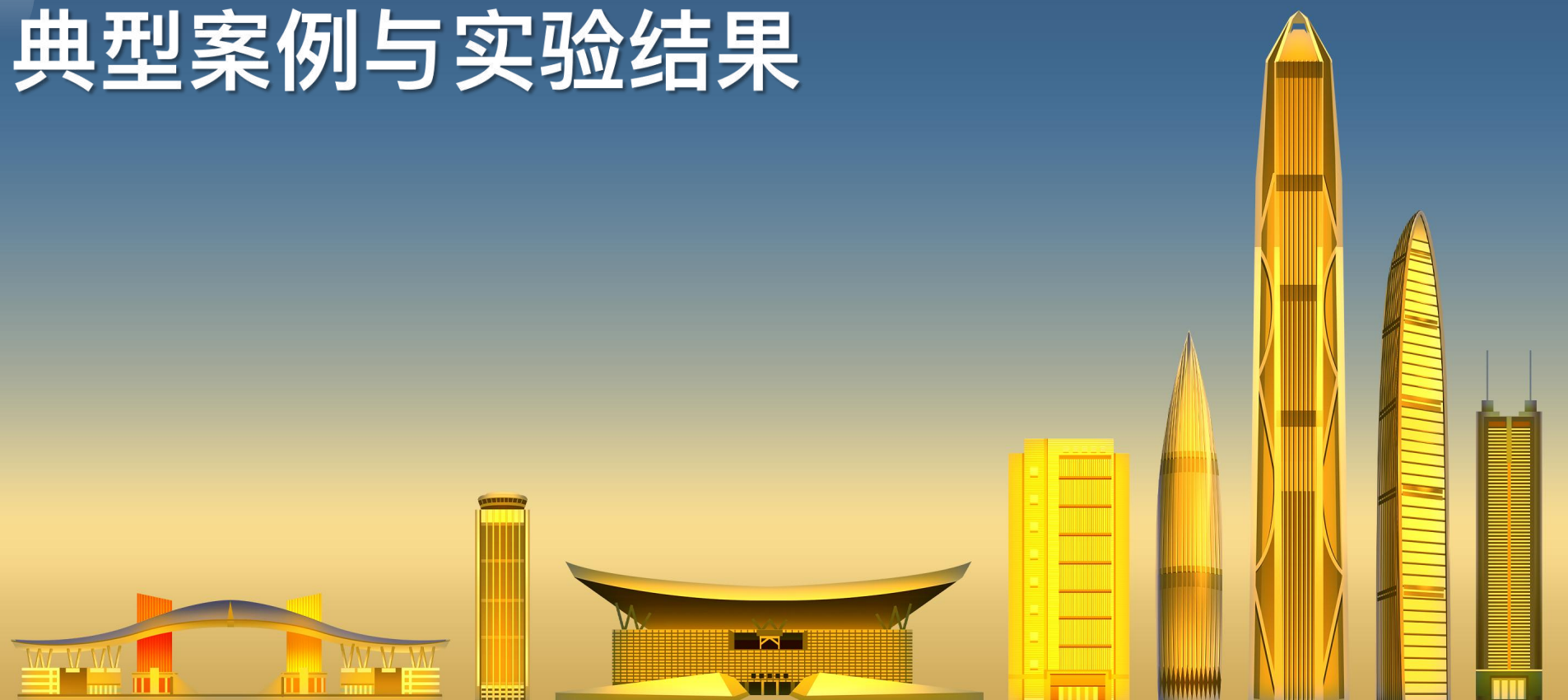
规格版本与变更影响分析
按风险设置人工检视门槛

持续集成与回归

需求关联测试接入 CI
积累回归集与验证证据

03

实践验证：典型案例与实验结果



案例一：“简历App”的开发需求

01 浏览分类

从资格证书编辑入口进入



1 默认展示“推荐证书”分类

02 搜索并高亮

输入“会计”，实时展示匹配证书



1 完整关键词实时筛选证书

2 命中的“会计”文字蓝色高亮

03 选择并退出搜索

点击“初级会计职称”，返回会计类

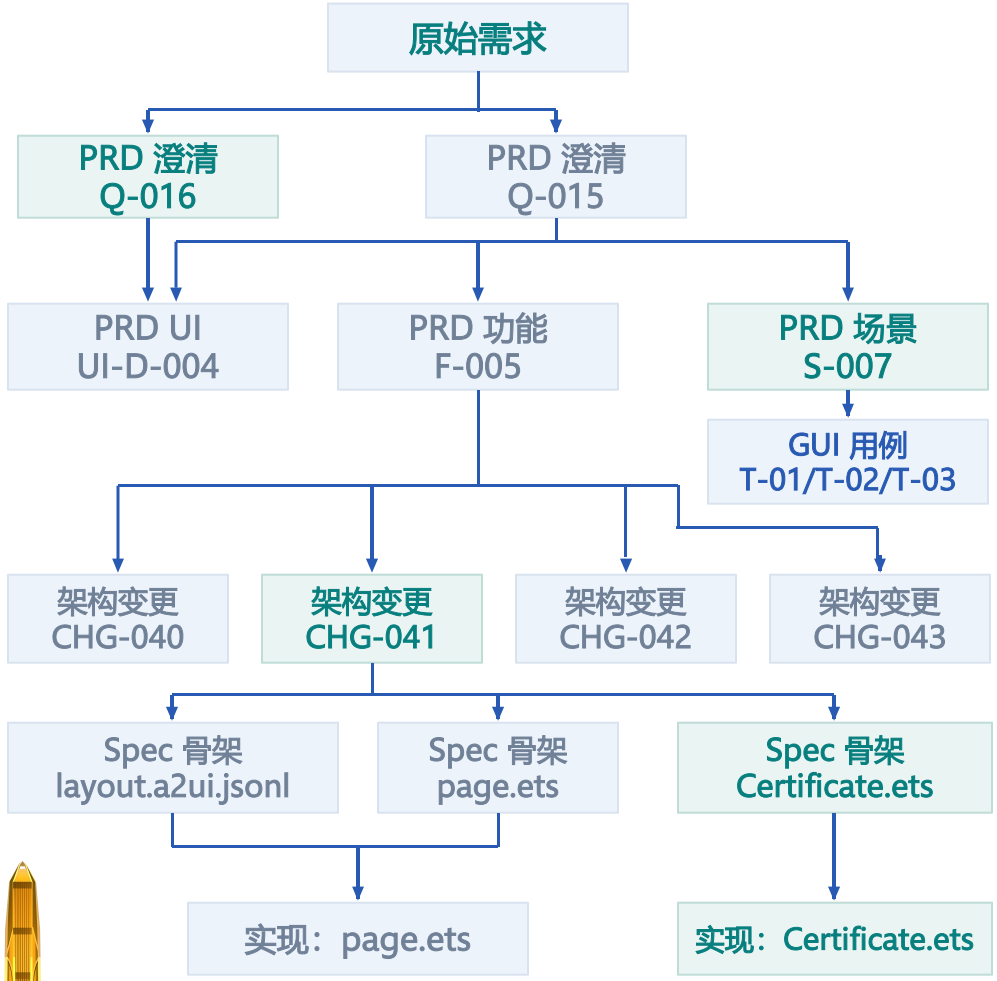


1 **清空搜索并退出搜索模式**

2 选中证书　　3 切换到会计类

案例一：搜索需求如何追踪到实现

01 追踪链：同一意图关联多份产物



02 证据：匹配规则与选择后行为

原始意图：实时搜索与命中高亮

输入关键词，实时筛选证书名称；英文忽略大小写，命中文字蓝色高亮。

Q-016：确定搜索匹配规则

问题摘要	当前材料片段	结论
搜索匹配规则细节（别名/拼音/多关键词/触发时机）？	「搜索不区分英文字母大小写」	按证书名称包含匹配，英文不区分大小写；输入作为一个完整关键词，不支持空格分词、拼音或别名匹配（用户逐项确认 2026-09-21）

S-007：明确搜索时的可见结果

GIVEN	已在资格证书选择页 P-002 的左右两栏模式
WHEN	在 cert-search-input 输入关键词
THEN	cert-result-list 实时替换两栏，显示名称包含关键词的证书
AND	命中文字高亮，匹配规则继承澄清结论

CHG-041：选择结果后退出搜索

```
@requires name != ''
@follows Checkbox(id=certificateSelect.UI-CERT-RESULT-CHECK).toggle
@ensures this.searchKeyword == '' && this.searchMode == false
// CHG: CHG-041
pickSearchResult(name: string): void
```

案例二：子 Agent 在实现前拦截需求遗漏

同一句占位提示，在需求到代码设计的传递中丢失

01 原始需求：已明确

搜索框占位提示为

“请搜索证书”

02 UI 布局：已保留

cert-search-input 已记录

“请搜索证书”

传递时遗漏

03 代码设计：遗漏

职责仅描述返回入口与搜索框

未写占位提示

原始需求中的界面参照



需求摘录

页面标题为“资格证书”，顶部提供返回入口和搜索框，占位提示为“请搜索证书”，页面提供“保存”按钮。

UI 布局记录：占位提示已保留

元素编号	页面	类型	动作	位置 / 布局	文案 / 资源	可见性
cert-page-title	P-002	页面标题	新增	顶部	资格证书	始终显示
cert-back-btn	P-002	返回入口	新增	顶部左侧	返回图标样式（随设计稿）	始终显示
cert-search-input	**P-002**	**搜索框**	**新增**	**顶部**	**请搜索证书（占位提示）**	**始终显示**

代码设计：子 Agent 对照检查，补齐遗漏

页面组件：QualificationCertificatePage（本次待实现）

修正前：职责缺项

顶部返回入口与搜索框……
（未描述占位提示）

→

修正后：补齐明确文案

顶部返回入口与搜索框……
占位提示为“请搜索证书”

实验结果：规格与反馈分别带来哪些改善

XX音乐-鸿蒙 App：13 项自然语言需求；下表展示部分结果

+19.3 pp

歌曲播放：77.2% 对比 57.9%

01 规格驱动开发与直接 Prompt

Cursor + composer-2.5

同一配置各运行 3 轮，展示平均通过率

需求	Prompt	SpecArtisans	差值 / pp
歌曲播放	57.9%	77.2%	+19.3
专辑	61.1%	78.6%	+17.5
歌单	68.4%	78.9%	+10.5
扫描音乐	96.3%	96.3%	0.0
统计	97.5%	96.3%	-1.2

规格的收益因任务而异，扫描音乐持平，统计略降

0/11 至 10/11

歌曲更多菜单：反馈后通过率达 90.9%

02 GUI 反馈引导定向修复

opencode + GLM-5.3-flash

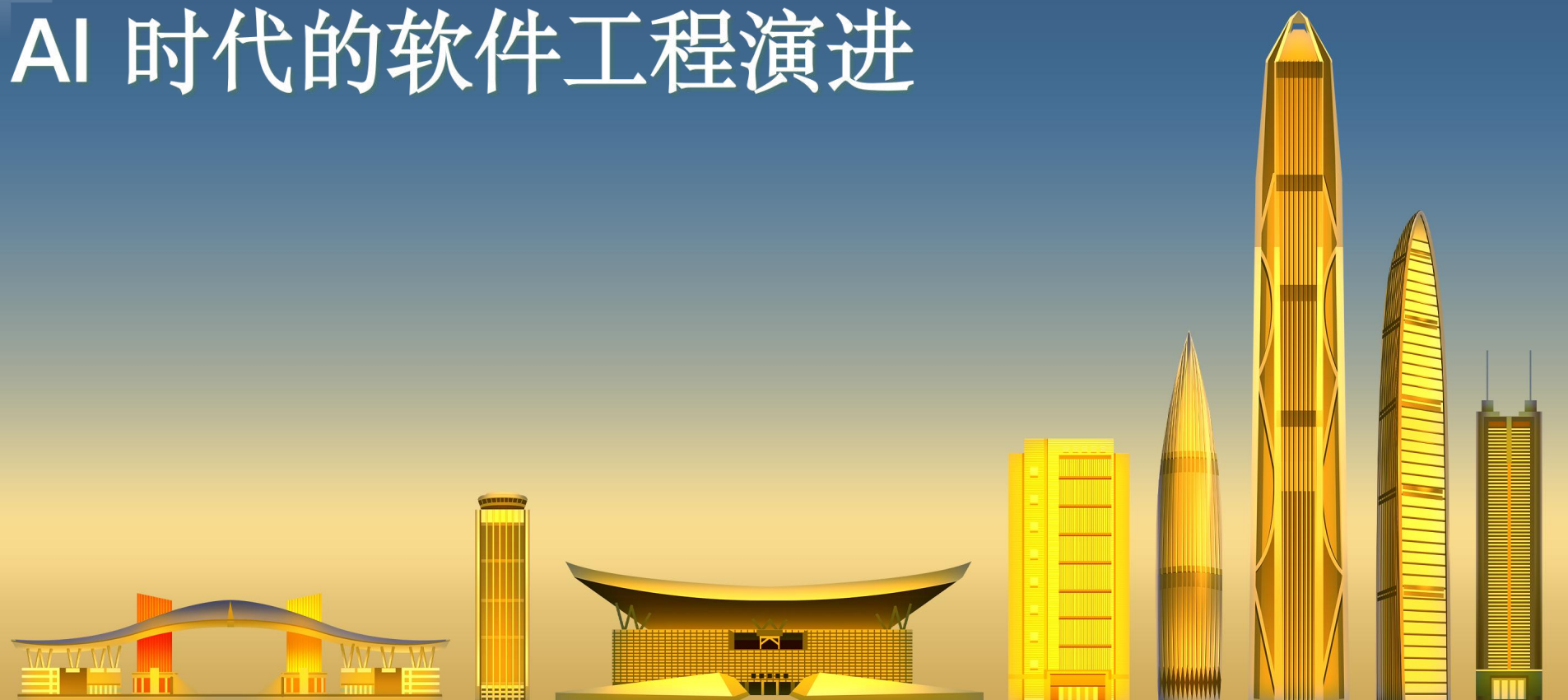
同一任务修复前后的通过数 / 检查点总数

需求	修复前	修复后	差值 / pp
歌曲更多菜单	0/11 (0%)	10/11 (90.9%)	+90.9
歌曲播放	2/19 (10.5%)	13/19 (68.4%)	+57.9
全屏播放器	34/43 (79.1%)	41/43 (95.3%)	+16.3
歌曲主页	47/49 (95.9%)	49/49 (100%)	+4.1
扫描音乐	16/18 (88.9%)	16/18 (88.9%)	0.0

反馈修复改善多项结果。扫描音乐仍有未通过项，主要原因是大模型无法调用正确的鸿蒙系统接口 API

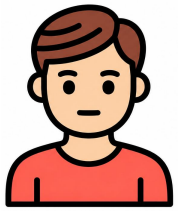
04

总结展望：AI 时代的软件工程演进



Spec 让人和 Agent 有据可依地协作

人决定目标与取舍，Spec 明确协作依据，Agent 自主执行与验证



人：关键判断

澄清真实需求与业务边界
决定架构取舍与变更范围
按风险介入，确认验收结果

表达与修订

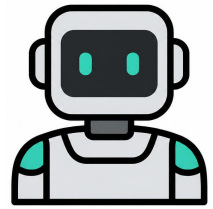
检视与决策

Spec：共同协作依据

PRD 与用户场景
架构变更与行为契约
验收标准与验证证据

约束实现

反馈证据



Agent：自主执行

理解存量架构，协同产码
执行用户旅程与一致性检查
依据偏差证据定向修复

支撑协作的四项机制

全链路追踪

需求、设计、代码与测试关联
过程可检视，偏差可回溯

Spec 双向闭环

规格驱动实现，实测反馈修复
Code-to-Spec 辅助一致性检查

架构感知增量

复用既有能力，约束修改范围
让新增功能融入存量系统

前后端一体化

统一场景与 API 契约
协同落实 UI、状态与业务逻辑

更高的自主性，需要更强的可验证性与可介入能力

生成能力越强，目标定义与可信验证越重要

我们的判断：软件开发的重心将更多转向“该做什么”与“如何确认做对”

开发者：定义值得解决的问题

目标与取舍

理解业务与真实世界，决定目标、约束和代价。
AI 可以搜索方案，价值判断仍需人承担责任。

验收与演进

设计评价标准，识别指标之外的风险。
在系统持续变化时，决定何时接受、何时介入。

编程语言：提供可验证的语义

让意图与边界可计算

以类型、契约与效应系统表达约束，
让人与 AI 共享精确定义的行为边界。

让生成与演化有证据

将验证器与编译器纳入生成过程，
探索携带证明的代码与保持语义的自动演化。

长期研究方向：面向 AI 的语言与工具链，如何让自主生成的软件持续可信？

GOSIM

GOSIM Shenzhen 2026

Thanks!

项目仓库：https://gitcode.com/Cangjie-SIG/SpecLang



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App

